Medical Software Development

Master Medical Informatics

Ronald Tanner, David Herzig

FHNW Life Science Technologies, Muttenz

March 2025

This chapter will cover methods how to control a project. Stakeholders measure projects by how well they are executed within the project constraints or baselines. A project baseline is an approved plan for a portion of a project. It is used to compare actual performance to planned performance and to determine if project performance is within acceptable guidelines.
The following 4 project baselines should be considered:

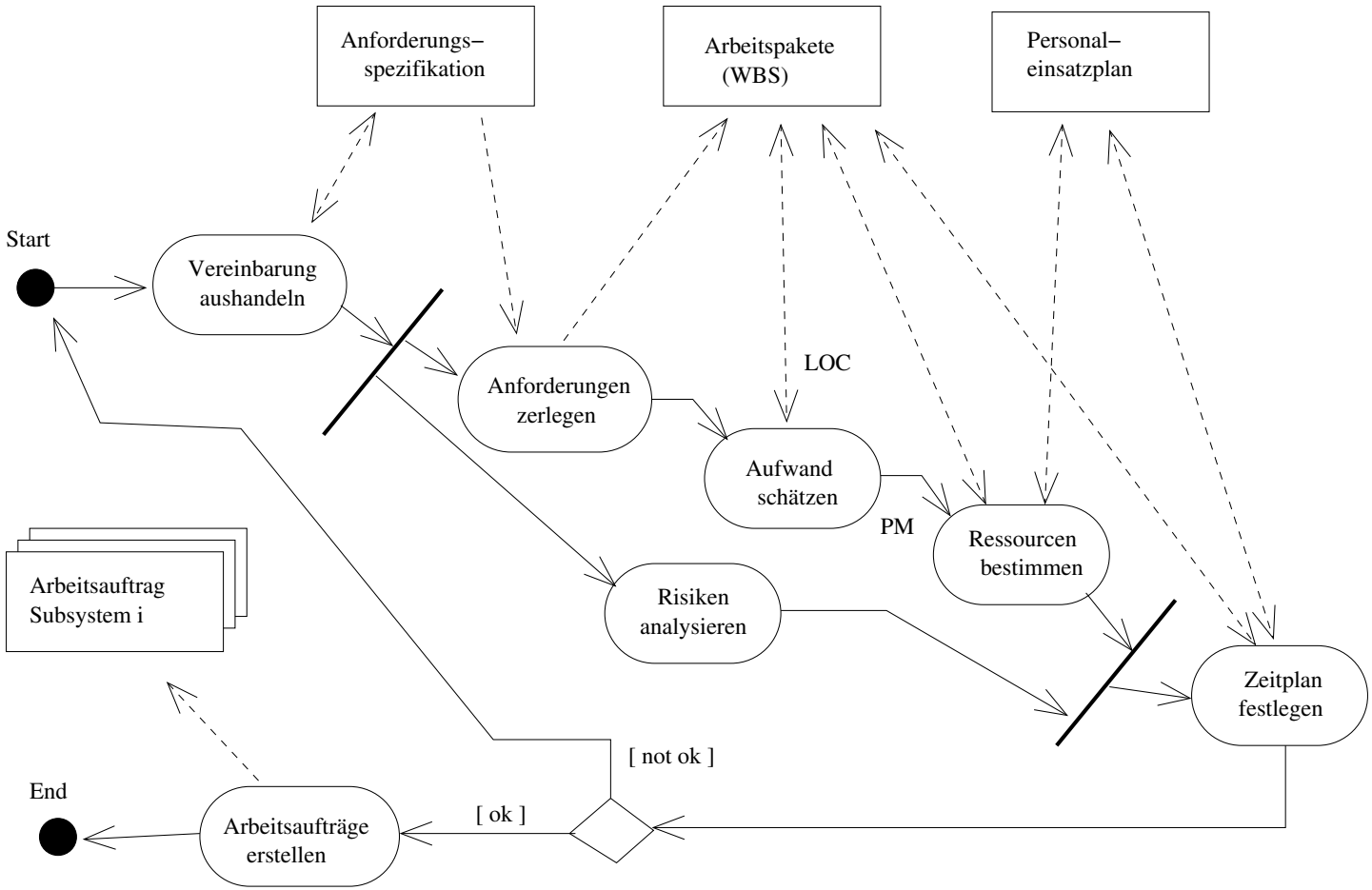
- Quality
- Schedule
- Scope
- Budget

Project Planning

Results:

- **C**ost plan: Creating budget / offer.
- **T**ime plan: Define activities, deadlines and milestones.
- **E**mployee plan: Nomination of people and their working time.
- **O**rganization plan: Agreement of team structure, nomination of responsible persons.
- **Q**uality plan: Compilation of documents, tools and methods to ensure quality.
- **P**roject monitor plan: Agreement of actions to control the project and if needed how to change the plans.
- **C**onfiguration management plan: Agreement of actions to control changes in documents, code or data (or any other resources).
- **E**ducation plan: Definition of education for internal (project team) and external people (users, operating stuff).
- **R**isk management plan: Agreement of actions to avoid or mitigate risks.

Classical Project Project Planning



Agile Project Planning

Software development is a constant dialog between *What is possible* and *What is needed*.

- Planning: only plan for the next iteration, version. Not more!
- Responsibility will be taken over, not assigned
- Estimates: the responsible persons define the effort and duration
- Priorities define the order

The business side (Product Owner) decides

- MVP: Minimal viable product. How should the problem be solved to bring additional value. What is to much, what is not enough?
- Priority: Which tasks should be executed first?
- Release: Which pieces should be part in the next release?
- Delivery: Which timeframe is available to be successful?

The developers have the freedom to take over responsibility and decide for

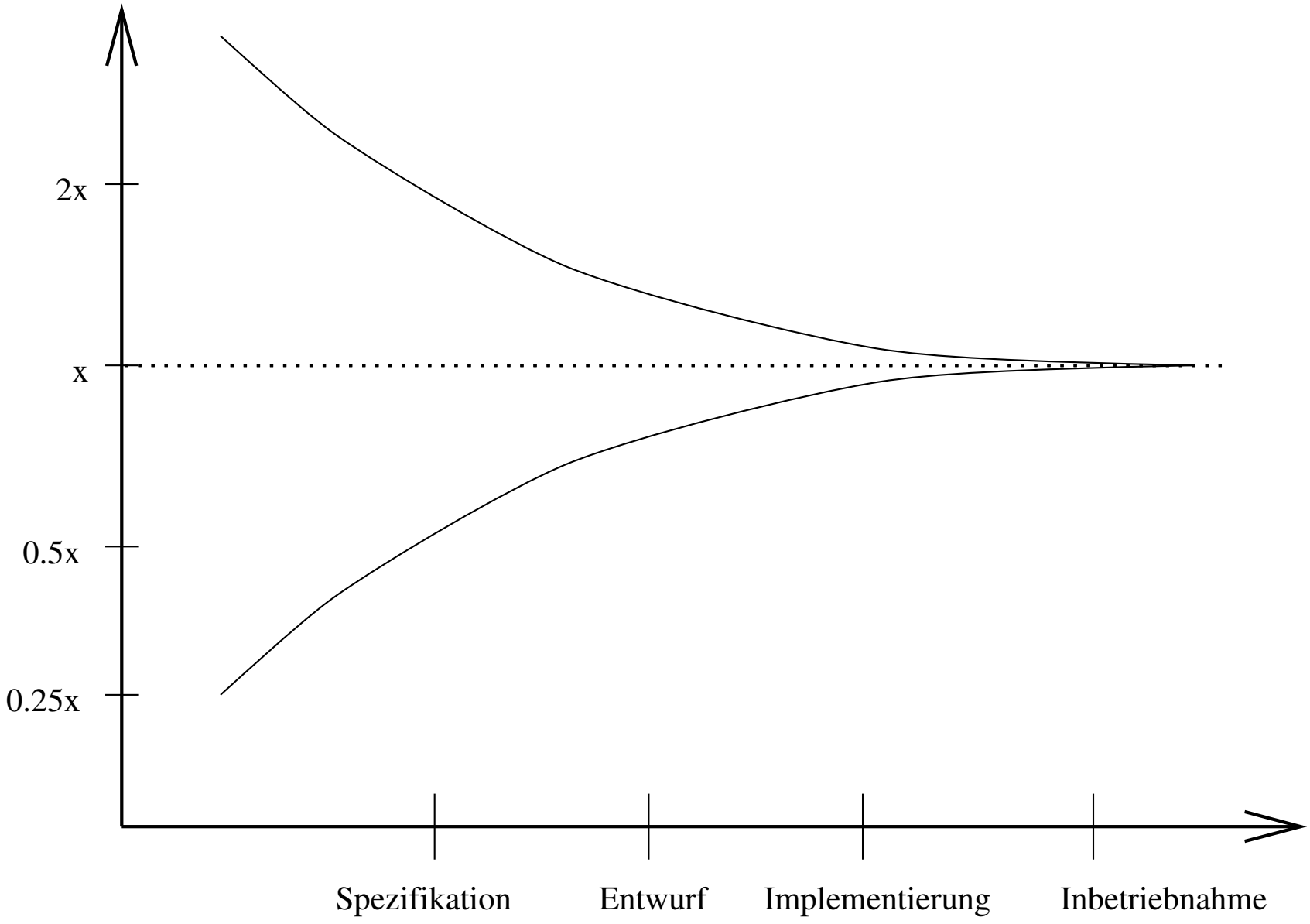
- Effort: how long will it take to implement a specific feature?
- Consequences: what are the consequences for a taken approach?
- Process: what is the structure for the work and the team?
- Planning: when will the features be delivered

Cost Estimation

The total costs of software projects contains

- **S**oftware-Cost: software, licenses
- **H**ardware-Cost: infrastructure (own hardware, cloud-based systems)
- **P**ersonal-Cost: Salary, Expenses

Cost Estimation Accuracy



Dominating Cost Drivers

Influencing factors to estimate the costs:

- **S**ize: Lines of code, classes, methods
- **C**omplexity: Dependency between the modules, is it possible to build modules?, HW/SW environment
- **E**xperience: Number of similar executed projects
- **R**eliability: error rate, system stability

Estimation Methods

- Empiric estimation methods:
  - **Expert judgment:** Several experts on the proposed software development techniques and the application domain are consulted. They each estimate the project cost. These estimates are compared and discussed. The estimation process iterates until an agreed estimate is reached.
  - **Delphi method:** Delphi technique is quite an old but efficient forecasting method. It follows an interactive approach which relies on exchange of ideas. The team is composed of a group of experts in their respective domains, who answers the queries in two or more rounds. Every time a facilitator provides a summary of the collected ideas, which is revised by the experts if required. The process of opinion and revaluation goes on until a final consensus is reached.
    Delphi technique relies its assumption on the fact that assimilation of ideas from a structured group leads to a productive outcome.

Estimation Methods (2)

- Algoritmic estimation methods:
  - **Function Point Analysis:** Function Point Analysis is a standardized method used commonly as an estimation technique in software engineering.
    In simple words, FPA is a technique used to measure software requirements based on the different functions that the requirement can be split into. Each function is assigned with some points based on the FPA rules and then these points are summarized using the FPA formula. The final figure shows the total man-hours required to achieve the complete requirement.
  - **Widget Point Analysis:** Count the UI (user interface) elements and calculate out of this number the function points. Only useful for software with user interfaces and not many algorithmic calculations in the background.
  - **Lines of Code (LOC):** Easy measurement method. Dependent on the programming language and the available libraries.

Estimation Methods (3)

- Other methods:
  - **Pricing to win** : The software cost is estimated to be whatever the customer has available to spend on the project. The estimated effort depends on the customer's budget and not on the software functionality.
  - **Pain threshold method**: Highest price which the customer is willing to pay
  - **Parkinson's Law**: Parkinson's Law states that work expands to fill the time available. The cost is determined by available resources rather than by objective assessment. If the software has to be delivered in 12 months and 5 people are available, the effort required is estimated to be 60 personmonths.

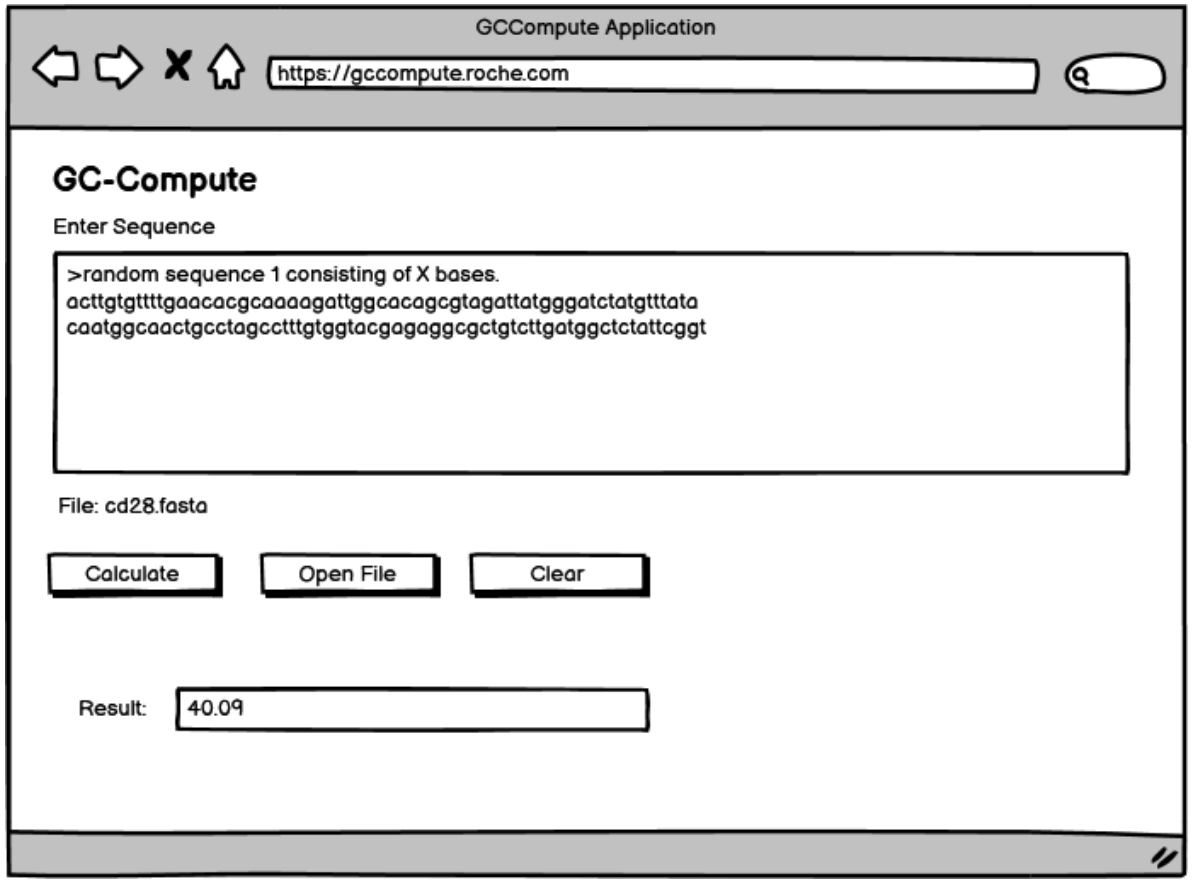
Function Points (FP) I

Function points are used to compute a functional size measurement (FSM) of software.: (www.ifpug.org)

Category	Number	Value (low) average (high)
External inputs		x (3) 4 (6)
External outputs		x (4) 5 (7)
External inquiries		x (3) 4 (6)
Internal logical files		x (7) 10 (15)
External interface files		x (5) 7 (10)
Total		

Function Points (FP) II

Example Function Points



Function Points (FP) III

Action/Process

1. Calculate
   - External Input: Sequence Data, 4
   - External Output: Result, 5
2. Open File
   - External Input: Filename, 4
   - External Output: Sequence Data and Filename, 5
3. Clear
   - External Output: Sequence Data (empty), 5
   - External Output: Result (empty), 5

Total number of Function Points: 28

FP Programming Languages Table

In a second step, the calculated function points could be converted into LOC (lines of code):

Programming language	LOC per FP
Assembler	320
C	128
Fortran	128
Pascal	91
C++/Java	53
SQL	13

Attention: this method does not reflect the re-usage of software components!

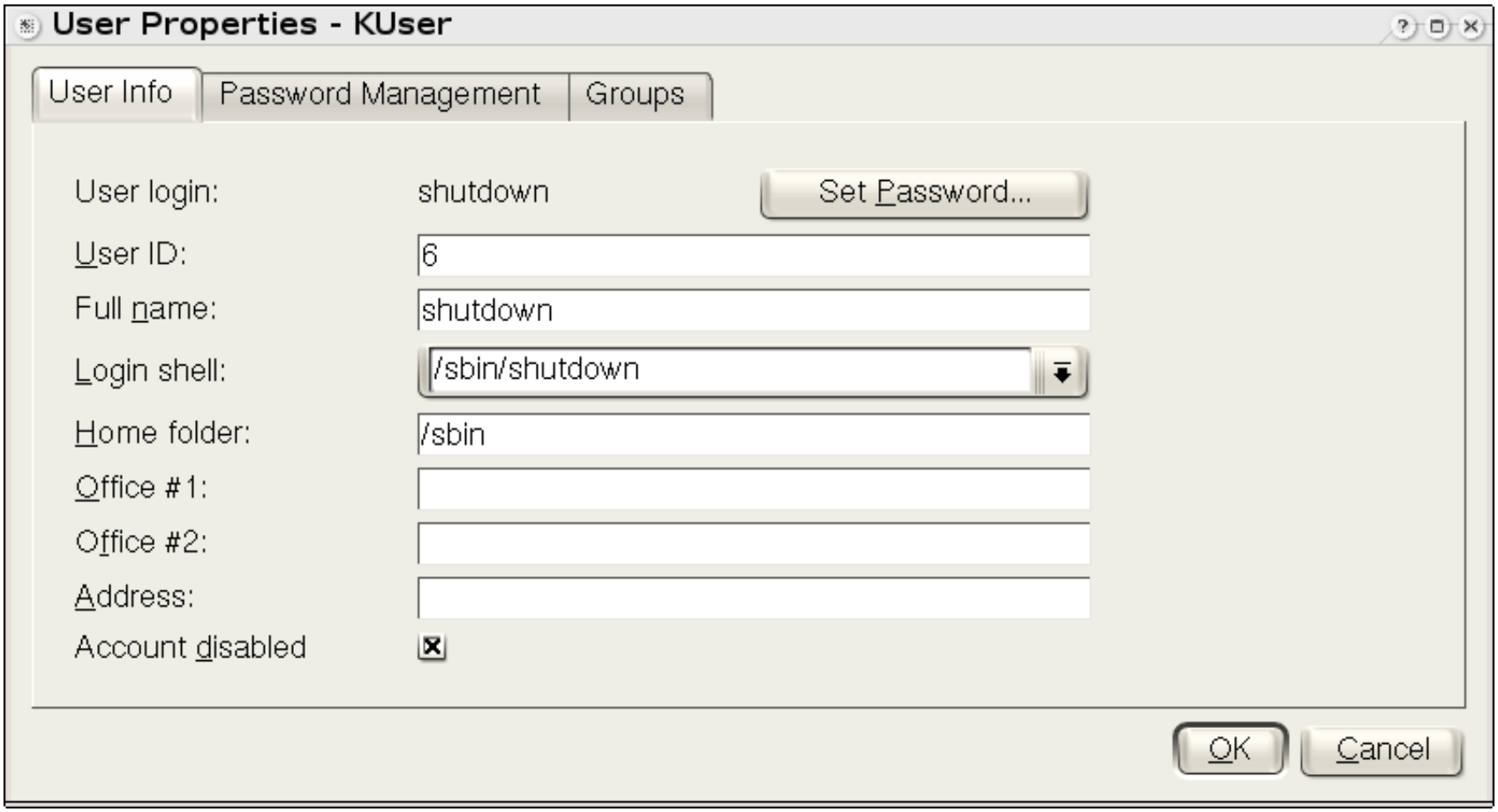
Widget-Points (H. Krasemann)

Calculate the number of function points based on the elements of the user interface.

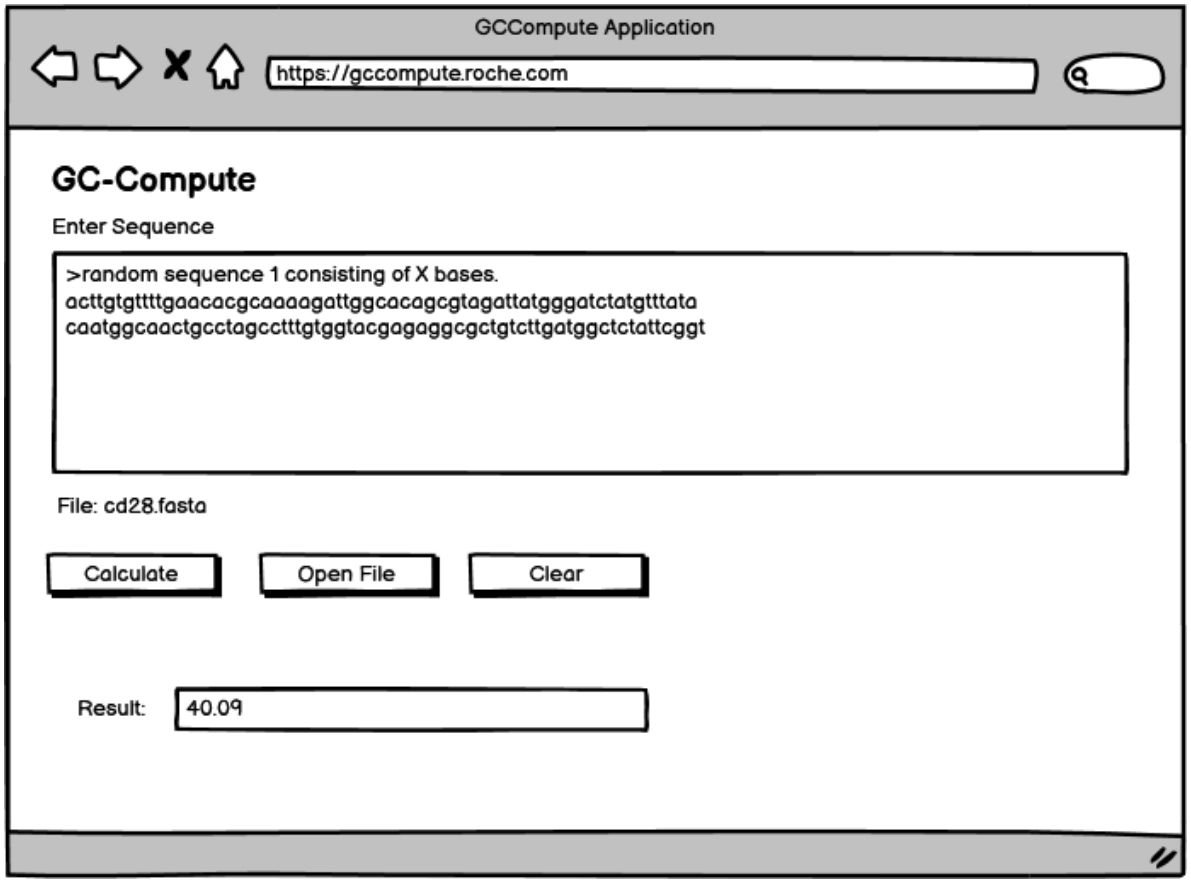
$$functionpoints = 2 \cdot widgetpoints \tag{1}$$

- **Input widgets:** Textfield, Combobox, Menubutton, Radiobutton, Pushbutton, Checkbox
- **Describing widgets:** Label, Separator, Group box, Window
- **Composite widgets:** Notebook, Table, Scrollbar, List
- **Menu widgets:** Menu bars, Men items

Widget Points II



Widget Points III



Widgetpoints: Input widgets (5), Describing widgets (4), Composite widgets (0), Menu widgets (0)

Functionpoints: $9 \cdot 2 = 18$

COCOMO (COnstructive-COst-MOdel, B. Boehm)

1. Calculation of the effort in person month:

$$E_i = a \cdot KDL^b \tag{2}$$

with the factor a the exponent b and KDL as number of delivered lines of code (in thousand) (Kilo-delivered-lines).

Project type	a	b	c
organic (simple)	3.2	1.05	0.38
semi-detached (medium)	3.0	1.12	0.35
embedded (complex)	2.8	1.20	0.32

2. Calculation of the effort in person month:

$$E = EAF \cdot E_i \tag{3}$$

with the correction factor EAF (Effort-adjustement-factor).

COCOMO Effort Adjustment Factor

Cost factors	very low	low	nominal	high	very high
Required Software Reliability	0.75	0.88	1.00	1.15	1.40
Size of Application Database		0.94	1.00	1.08	1.16
Complexity of The Product	0.70	0.85	1.00	1.15	1.30
Runtime Performance Constraints			1.00	1.11	1.30
Memory Constraints			1.00	1.06	1.21
Response times		0.87	1.00	1.07	1.15
Analyst capability	1.46	1.19	1.00	0.86	0.71
Applications experience	1.29	1.13	1.00	0.91	0.82
Software engineer capability	1.42	1.17	1.00	0.86	0.70
Programming language experience	1.14	1.07	1.00	0.95	
Application of software engineering methods	1.24	1.10	1.00	0.91	0.82
Use of software tools	1.24	1.10	1.00	0.91	0.83
Required development schedule	1.23	1.08	1.00	1.04	1.10

COCOMO Effort Partitioning

Software size	small (2 KDL)	medium (32 KDL)	large (128 KDL)
Design	16%	16%	16%
Implementation	68%	62%	59%
Integration + Tests	16%	22%	25%

SCRUM: Story Points

Story points are a unit of measure for expressing an estimate of the overall effort that will be required to fully implement a product backlog item or any other piece of work. When we estimate with story points, we assign a point value to each item. The raw values we assign are unimportant.

The Fibonacci sequence is one popular scoring scale for estimating agile story points. In this sequence, each number is the sum of the previous two in the series.
0, 1, 2, 3, 5, 8, 13, 21....

SCRUM: Story Points

Also, it's useful to set a maximum limit (13, for instance). If a task is estimated to be greater than that limit, it should be split into smaller items. Similarly, if a task is smaller than 1, it should be incorporated into another task.

Before each estimation round, set 2 reference points: two good predictable stories, one with 2 the other one with 5 points.

Burn-Down Chart

A burndown chart shows the amount of work that has been completed in an epic or sprint, and the total work remaining. Burndown charts are used to predict your team's likelihood of completing their work in the time available.

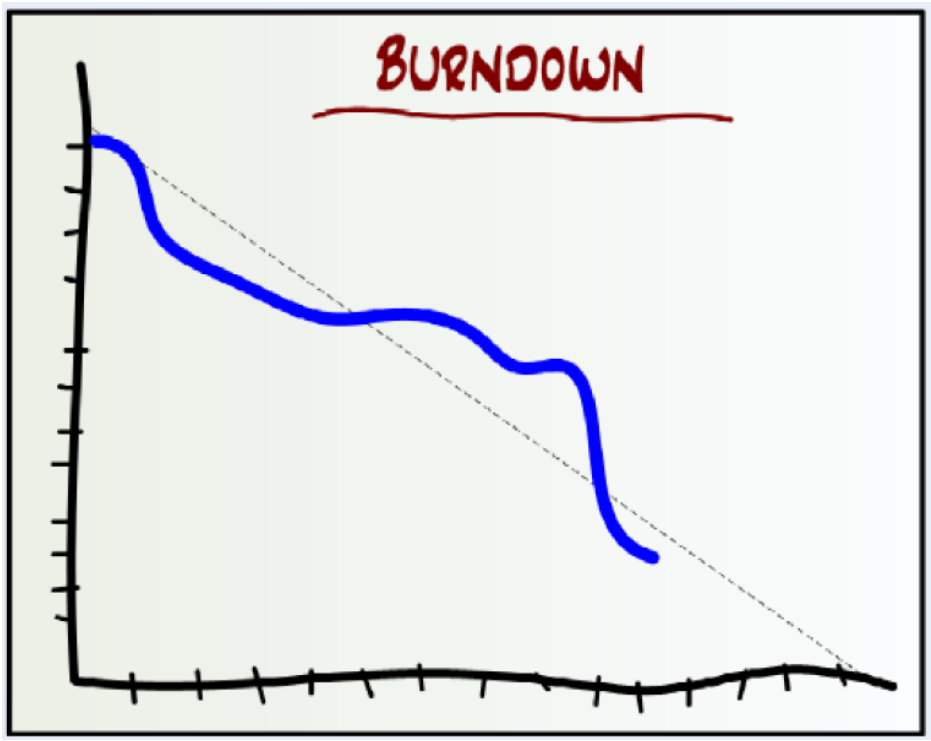


Figure: Burndown chart (Quelle: Henrik Kniberg)

And what about Microsoft Project?

Software development guru Joel Spolsky puts it this way:

> *The trouble with Microsoft Project is that it assumes that you want to spend a lot of time worrying about dependencies... I've found that with software, the dependencies are so obvious that it's just not worth the effort to formally keep track of them. Another problem with Project is that it assumes that you're going to want to be able to press a little button and "rebalance" the schedule... For software, this just doesn't make sense [in practice]... The bottom line is that Project is designed for building office buildings, not software.*

Joel is a former Microsoft employee, who worked on both Excel and Project.

Project Schedule Plan

Axiom: persons and months are NOT exchangeable!

1. Calculate the length of a project in months based on the calculated person months:

$$D = 2.5 \cdot E^{c}$$
$$Organic: D = 2.5 \cdot E^{0.38} \tag{4}$$

2. Caclculate the average demand of employees:

$$P = \frac{E}{D} \tag{5}$$

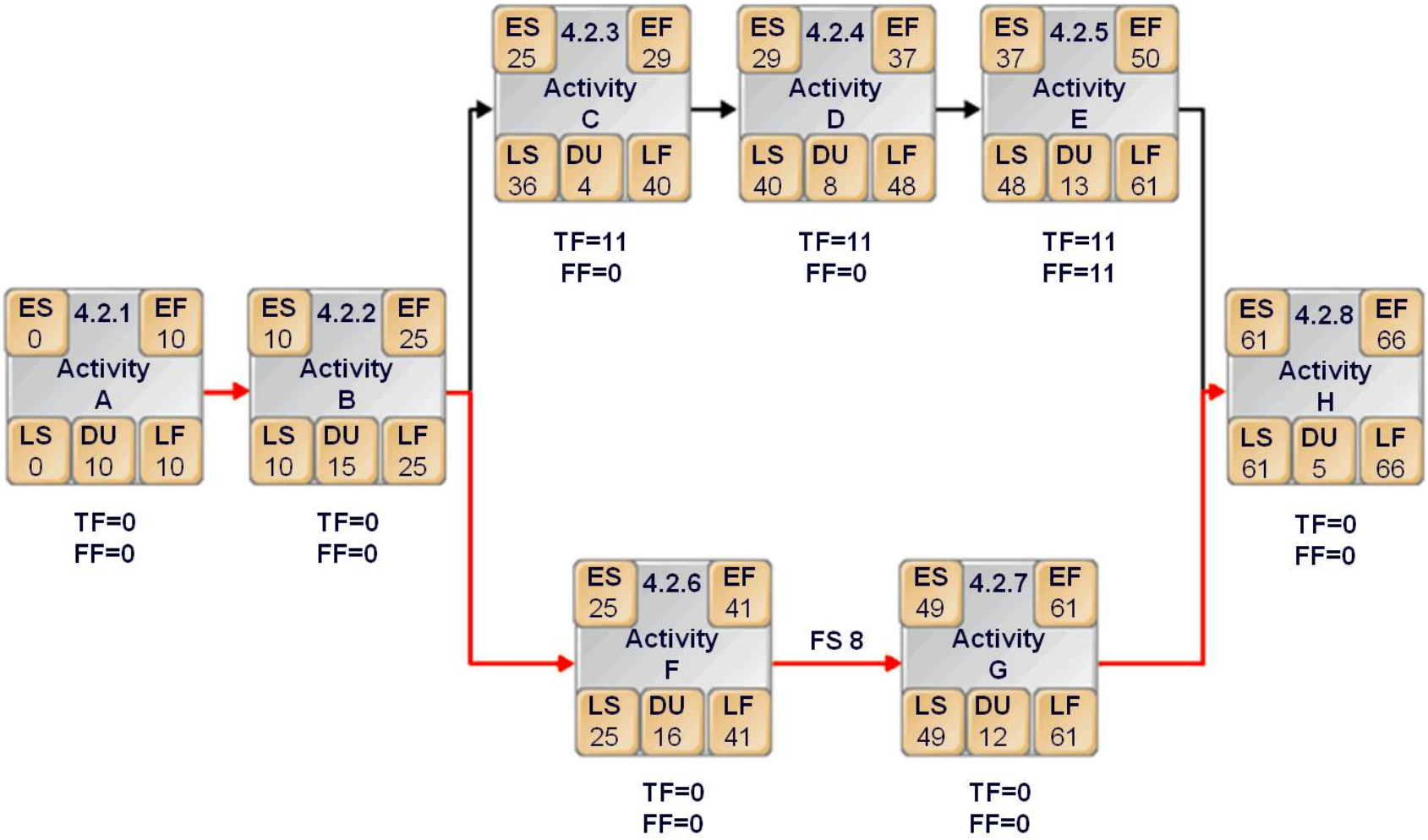
3. Calculation of the effort distribution (in percentage):

Software size	small (2 KDL)	medium (32 KDL)	large (128 KDL)
Design	19	19	19
Implementation	63	55	51
Integration + Tests	18	26	30

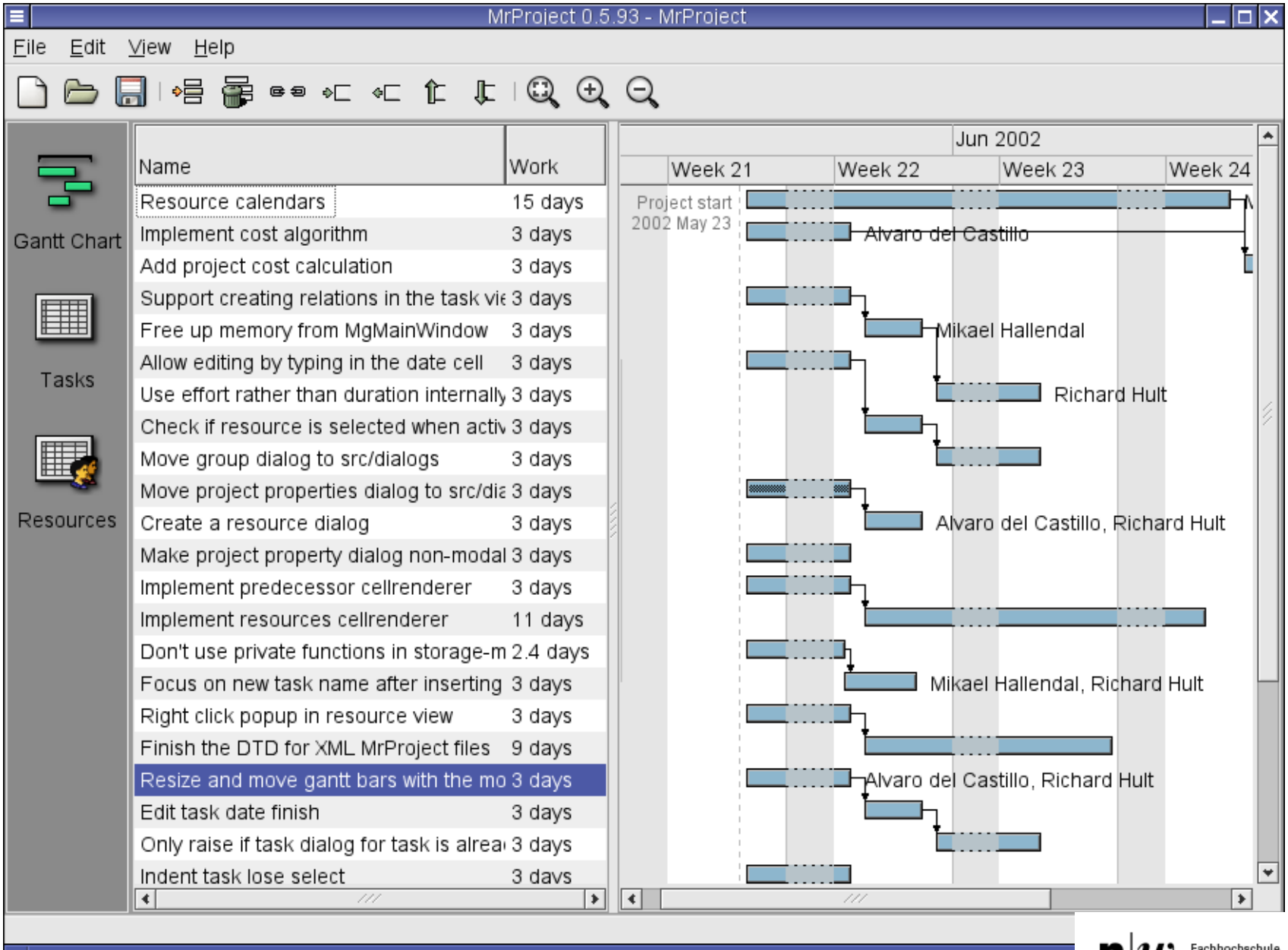
4. Refinement of all phases and creation of the predecessor list:

No	Task	Predecessor	Duration	Ressource
1	Software design		5	Meier
2	Design approval	1	1	(Review)
3	Implementation GUI	2	25	Hilfiker
4	Implementation DB	2	12	...
5	Module test GUI	3	3	...
6	Module test DB	4	2	...
7	Integration	5,6	2	...

Critical Path Method / Critical Path Analysis



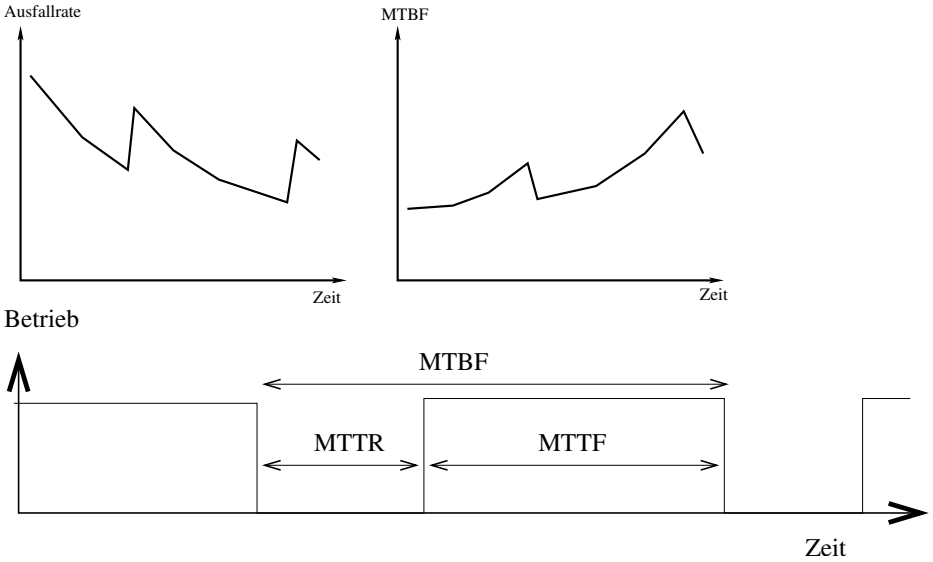
Gantt Diagram



Measurements and Metrics

Attribute	Measurement
Software size	(LOC) Number of lines of code
Complexity	Number of classes, methods, relations
Change frequency	Number of changes in a given time frame
Fault rate	Number of errors in a given time frame
Productivity	Number of lines in a given time frame
Effort	Number of working hours

Failures, Bugs and Availability I



failure	Deviation of the operating behavior from its requirements
fault, bug	Cause of one or more malfunctions
error	wrong, incorrect or incomplete implementation
reliability	Probability of trouble-free operation during a certain period of time
availability	Ratio of the trouble-free operating time to the total operating time

Failures, Bugs and Availability II

MTBF	mean time between failures
MTTR	mean time to repair
MTTF	mean time to failure

Nines of Reliability:	downtime per year [h]	[min]	[sec]
2 9's (99%)	87.6	5256.0	315360.0
3 9's (99.9%)	8.76	525.6	31536.0
4 9's (99.99%)	0.876	52.56	3153.6

Heroku Uptime Metrics

Exercises I

1. What is the critical path for the given task list and what is the duration (of the critical path)?

Task	Duration (Days)	Predecessor
A	4	- (Start)
B	3	A
C	1	B
D	5	- (Start)
E	4	D
F	6	E
G (end)	2	C,F
H	4	D
I (end)	2	H

2. A new start up company needs to develop a software that is estimated with 850 function points and is planning to use Java as programming language. The project type is rated as semi-detached (medium). The software is dealing with a large amount of data and has therefore high memory constraints. In addition, the software is highly complex. How many person month does the project take to complete?